





# 1. Conception de l'interface

L'application cliente est une application React (Vite) monopage qui consomme l'API REST de la Phase 1, mise à jour pour la Phase 2. Elle s'articule autour des deux vues complémentaires exigées par l'énoncé : une vue carte qui donne une lecture d'ensemble de l'ambiance des lieux instrumentés, et une vue détaillée qui dresse le portrait complet d'un lieu (classification courante, historique, créneaux calmes, contribution d'observations).

## 1.1 Décomposition en composants

L'interface est découpée en composants à responsabilité unique, conformément à l'exigence d'organisation modulaire :

| Composant                        | Responsabilité                                                                                                                                                                 |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App.jsx                          | État global : lieu sélectionné, session utilisateur (token JWT), favoris, navigation entre vues.                                                                               |
| MapView.jsx                      | Carte Leaflet/OpenStreetMap : marqueurs positionnés par coordonnées, colorés selon la classification, popup résumé, clic vers la vue détaillée, légende.                       |
| LocationDetail.jsx               | Portrait d'un lieu : badge de classification, graphe d'historique (Chart.js), créneaux calmes, formulaire d'observation (si connecté), bouton favori.                          |
| LoginForm.jsx / RegisterForm.jsx | Formulaires de connexion et de création de compte, avec gestion des erreurs renvoyées par l'API.                                                                               |
| MyLocations.jsx                  | Vue « Mes lieux » : récapitulatif des lieux où l'utilisateur connecté a soumis des observations (nombre d'écoutes, dernière contribution, accès au détail, gestion du favori). |
| api/ambianceApi.js               | Couche client : seule zone du code qui parle à l'API (axios). Aucun composant n'appelle l'API directement.                                                                     |

## 1.2 Parcours utilisateur

À l'ouverture, l'utilisateur voit la carte des lieux et leur ambiance courante sans aucune authentification (accès public). Un clic sur un marqueur ouvre le portrait détaillé du lieu. L'en-tête reflète en permanence l'état de session : bouton « Connexion » pour un visiteur, salutation, filtre « Mes favoris », accès à la vue « Mes lieux » et bouton « Déconnexion » pour un utilisateur connecté. Les actions protégées (soumettre une observation, gérer les favoris) ne sont affichées que si l'utilisateur est connecté, conformément à l'exigence d'affichage/masquage selon les permissions.

## 1.3 Gestion explicite des états

Chaque vue distingue explicitement trois états, comme exigé : chargement (indicateur « Chargement... » pendant les appels), erreur (message dédié si l'API est injoignable ou renvoie une erreur) et vide (« Aucune mesure disponible sur les dernières 24 heures » pour l'historique, « Aucun créneau calme détecté » pour les créneaux, marqueur gris « Données non disponibles » sur la carte).

## 2. Consommation de l'API

### 2.1 Couche client isolée

Tous les appels HTTP passent par `client/src/api/ambianceApi.js`. Cette couche centralise l'URL de base (configurable via la variable d'environnement `VITE_API_URL`, fichier `client/.env` — un `.env.example` est fourni), la construction des requêtes et l'injection de l'en-tête `Authorization: Bearer <token>` pour les actions protégées. Les composants ne connaissent ni `axios` ni les URLs : changer d'API ou de transport ne toucherait que ce fichier.

### 2.2 Endpoints consommés

| Endpoint                                                            | Usage dans le client                                                 | Auth     |
|---------------------------------------------------------------------|----------------------------------------------------------------------|----------|
| GET <code>/v1/locations</code>                                      | Liste des lieux et coordonnées pour la carte                         | publique |
| GET <code>/v1/ambiance/{slug}/now?window=30m</code>                 | Badge de classification (carte et vue détaillée)                     | publique |
| GET <code>/v1/ambiance/{slug}/history?last=24h&amp;bucket=1h</code> | Graphe d'historique du niveau sonore                                 | publique |
| GET <code>/v1/ambiance/{slug}/quiet-hours?days=7</code>             | Créneaux calmes de la semaine                                        | publique |
| POST <code>/v1/auth/register</code>                                 | Création de compte                                                   | publique |
| POST <code>/v1/auth/login</code>                                    | Connexion (obtention du JWT)                                         | publique |
| GET/POST/DELETE <code>/v1/auth/favorites</code>                     | Consultation et gestion des lieux favoris                            | JWT      |
| GET <code>/v1/auth/my-locations</code>                              | Vue « Mes lieux » : lieux où l'utilisateur a soumis des observations | JWT      |
| POST <code>/v1/observations/user</code>                             | Soumission d'une observation liée à l'auteur                         | JWT      |

### 2.3 Format des réponses et gestion des erreurs

L'API renvoie une enveloppe uniforme : `{ status, data, meta }` en succès et `{ status: "error", error: { code, message, details? }, meta }` en erreur. Les messages d'erreur sont descriptifs et en français, ce qui permet au client de les afficher tels quels sans documentation externe (exigence de réponses auto-descriptives). Côté client, chaque appel est encadré : les erreurs réseau ou HTTP alimentent l'état erreur de la vue, et les erreurs de validation des formulaires affichent `error.message` renvoyé par le serveur (ex. champ manquant, identifiants invalides, valeur hors domaine). Les codes utilisés : 400 (validation), 401 (token absent/expiré), 403 (interdit), 404 (lieu inconnu), 409 (conflit à l'inscription), 422 (valeur hors domaine).

## 3. Visualisation

L'authentification utilisateur repose sur des tokens JWT signés côté serveur (JWT\_SECRET, expiration 7 jours). Les mots de passe sont hachés avec bcrypt avant stockage (jamais en clair), avec une longueur minimale de 6 caractères validée à l'inscription. La connexion accepte le nom d'utilisateur ou l'adresse courriel. Le token et le profil sont conservés dans le localStorage pour maintenir la session entre les visites ; la déconnexion les efface.

## 4.2 Actions protégées

Le middleware serveur userAuth vérifie l'en-tête Authorization: Bearer <token> et renvoie 401 (NO\_TOKEN / INVALID\_TOKEN) si le token est absent, invalide ou expiré. Sont protégés : la soumission d'observation (POST /v1/observations/user, qui enregistre automatiquement l'utilisateur comme author et horodate côté serveur) et la gestion des favoris. Côté client, ces actions sont masquées pour les visiteurs non connectés ; l'authentification device de la Phase 1 (x-api-key) reste inchangée pour la collecte capteur.

## 4.3 Espace compte

Une fois connecté, l'utilisateur dispose de : la création/suppression de favoris depuis la vue détaillée, un filtre « Mes favoris » sur la carte pour retrouver ses lieux personnels, le formulaire d'observation (densité, proximité, ambiance ressentie, notes) avec confirmation de soumission et affichage des erreurs de validation du serveur, et la vue « Mes lieux » : un récapitulatif des lieux où il a effectué des écoutes, construit côté serveur (GET /v1/auth/my-locations, agrégation MongoDB des observations par lieu) qui affiche pour chaque lieu le nombre d'observations soumises, la date de la dernière contribution, le statut favori (modifiable sur place) et un accès direct au portrait détaillé. La liaison observation@auteur ajoutée à l'infrastructure (section 5) est ainsi exploitée de bout en bout.

# 5. Mise à jour de l'infrastructure (API Phase 1)

La contrainte directrice était de faire évoluer sans rompre : aucun endpoint de la Phase 1 n'a été modifié ni supprimé, et tous les nouveaux champs sont optionnels. La collecte par bridge Phyphox et les lectures existantes fonctionnent à l'identique.

| Évolution               | Détail                                                                                               | Rétrocompatibilité                                                   |
|-------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Coordonnées des lieux   | Champs latitude/longitude (validés -90..90 / -180..180) sur Location, exposés par GET /v1/locations. | Champs optionnels : les lieux existants restent valides.             |
| Auteur des observations | Champ author (référence User) sur Observation, renseigné par POST /v1/observations/user.             | Optionnel : les observations device (x-api-key) restent sans auteur. |
| Modèle User (nouveau)   | username et email uniques, mot de passe bcrypt, favoriteLocations.                                   | Nouvelle collection : aucun impact sur l'existant.                   |
| Routes /v1/auth/*       | register, login, gestion des favoris (JWT).                                                          | Nouvelles routes montées à côté des anciennes.                       |

L'authentification utilisateur repose sur des tokens JWT signés côté serveur (JWT\_SECRET, expiration 7 jours). Les mots de passe sont hachés avec bcrypt avant stockage (jamais en clair), avec une longueur minimale de 6 caractères validée à l'inscription. La connexion accepte le nom d'utilisateur ou l'adresse courriel. Le token et le profil sont conservés dans le localStorage pour maintenir la session entre les visites ; la déconnexion les efface.

## 4.2 Actions protégées

Le middleware serveur userAuth vérifie l'en-tête Authorization: Bearer <token> et renvoie 401 (NO\_TOKEN / INVALID\_TOKEN) si le token est absent, invalide ou expiré. Sont protégés : la soumission d'observation (POST /v1/observations/user, qui enregistre automatiquement l'utilisateur comme author et horodate côté serveur) et la gestion des favoris. Côté client, ces actions sont masquées pour les visiteurs non connectés ; l'authentification device de la Phase 1 (x-api-key) reste inchangée pour la collecte capteur.

## 4.3 Espace compte

Une fois connecté, l'utilisateur dispose de : la création/suppression de favoris depuis la vue détaillée, un filtre « Mes favoris » sur la carte pour retrouver ses lieux personnels, le formulaire d'observation (densité, proximité, ambiance ressentie, notes) avec confirmation de soumission et affichage des erreurs de validation du serveur, et la vue « Mes lieux » : un récapitulatif des lieux où il a effectué des écoutes, construit côté serveur (GET /v1/auth/my-locations, agrégation MongoDB des observations par lieu) qui affiche pour chaque lieu le nombre d'observations soumises, la date de la dernière contribution, le statut favori (modifiable sur place) et un accès direct au portrait détaillé. La liaison observation@auteur ajoutée à l'infrastructure (section 5) est ainsi exploitée de bout en bout.

# 5. Mise à jour de l'infrastructure (API Phase 1)

La contrainte directrice était de faire évoluer sans rompre : aucun endpoint de la Phase 1 n'a été modifié ni supprimé, et tous les nouveaux champs sont optionnels. La collecte par bridge Phyphox et les lectures existantes fonctionnent à l'identique.

| Évolution               | Détail                                                                                               | Rétrocompatibilité                                                   |
|-------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Coordonnées des lieux   | Champs latitude/longitude (validés -90..90 / -180..180) sur Location, exposés par GET /v1/locations. | Champs optionnels : les lieux existants restent valides.             |
| Auteur des observations | Champ author (référence User) sur Observation, renseigné par POST /v1/observations/user.             | Optionnel : les observations device (x-api-key) restent sans auteur. |
| Modèle User (nouveau)   | username et email uniques, mot de passe bcrypt, favoriteLocations.                                   | Nouvelle collection : aucun impact sur l'existant.                   |
| Routes /v1/auth/*       | register, login, gestion des favoris (JWT).                                                          | Nouvelles routes montées à côté des anciennes.                       |

## 7.2 Évolutions envisagées

- **Temps réel** : diffusion des nouvelles mesures par SSE ou WebSocket pour rafraîchir carte et graphe sans rechargement (bonus de l'énoncé).
- **Portage TypeScript du backend** : typage complet des modèles et des enveloppes de réponse (second bonus).
- **Espace compte enrichi** : étendre la vue « Mes lieux » avec la liste paginée des observations individuelles et des statistiques personnelles.
- **Comparaison de lieux** : exploiter `/v1/ambiance/compare` dans l'interface pour recommander le lieu le plus calme à un instant donné.
- **Affichage de la fraîcheur** : exposer `lastMeasurementAt` par lieu et afficher « mis à jour il y a X min » sur les marqueurs.